Build Array from Permutation

Aim :

To write a Java program to build a new array from a given permutation array using the rule ans[i] = nums[nums[i]]

Algorithm :

1. Start the program.
2. Create an integer array `nums`.
3. Use `IntStream.range()` to visit each index of the array.
4. For every index `i`, place `nums[nums[i]]` in the answer array.
5. Display the input and output arrays.
6. Stop the program.

Program :

```
import java.util.Arrays;
import java.util.stream.IntStream;

public class Main {
   public static void main(String[] args) {
      int[] nums = {0, 2, 1, 5, 3, 4};
      int[] ans = IntStream.range(0, nums.length).map(i -> nums[nums[i]]).toArray();
      System.out.println("Input: " + Arrays.toString(nums));
      System.out.println("Output: " + Arrays.toString(ans));
   }
}
```

Sample Output :

Input: [0, 2, 1, 5, 3, 4]

Output: [0, 1, 2, 4, 5, 3]

Shuffle the Array

Aim :

To write a Java program to shuffle an array in alternate order using map

Algorithm :

Step 1 Start the program

Step 2 Create an integer array nums

Step 3 Store the value of n

Step 4 Generate indexes from zero to two n minus one

Step 5 Pick values from the first half for even positions

Step 6 Pick values from the second half for odd positions

Step 7 Display the shuffled array

Step 8 Stop the program

Program :

```
import java.util.Arrays;
import java.util.stream.Collectors;
import java.util.stream.IntStream;

public class Main {
  static String join(int[] arr) {
    return Arrays.stream(arr).mapToObj(String::valueOf).collect(Collectors.joining(" "));
  }

  public static void main(String[] args) {
    int[] nums = {2, 5, 1, 3, 4, 7};
    int n = 3;
```

```
        int[] ans = IntStream.range(0, 2 * n)
                .map(i -> i % 2 == 0 ? nums[i / 2] : nums[n + i / 2])
                .toArray();
        System.out.println("Input " + join(nums));
        System.out.println("Output " + join(ans));
    }
}
```

Sample Output :

Input 2 5 1 3 4 7

Output 2 3 5 4 1 7

Remove Element

Aim :

To write a Java program to remove all occurrences of a given value from an array using filter

Algorithm :

Step 1 Start the program

Step 2 Create an integer array nums

Step 3 Store the value to be removed

Step 4 Convert the array into a stream

Step 5 Use filter to keep only values that are not equal to the given value

Step 6 Convert the result into an array

Step 7 Display the length and output array

Step 8 Stop the program

Program :

```
import java.util.Arrays;
import java.util.stream.Collectors;

public class Main {
  static String join(int[] arr) {
    return Arrays.stream(arr).mapToObj(String::valueOf).collect(Collectors.joining(" "));
  }

  public static void main(String[] args) {
    int[] nums = {3, 2, 2, 3};
    int val = 3;
```

```
        int[] result = Arrays.stream(nums).filter(x -> x != val).toArray();
        System.out.println("Value " + val);
        System.out.println("Length " + result.length);
        System.out.println("Output " + join(result));
    }
}
```

Output :

Value 3

Length 2

Output 2 2

Remove Duplicates from Sorted Array

Aim :

To write a Java program to remove duplicate elements from a sorted array using filter and stream processing

Algorithm :

Step 1 Start the program

Step 2 Create a sorted integer array with duplicate values

Step 3 Convert the array into a stream

Step 4 Use distinct to remove repeated values

Step 5 Store the unique values in a new array

Step 6 Display the length and unique array

Step 7 Stop the program

Program :

```
import java.util.Arrays;
import java.util.stream.Collectors;

public class Main {
  static String join(int[] arr) {
    return Arrays.stream(arr).mapToObj(String::valueOf).collect(Collectors.joining(" "));
  }

  public static void main(String[] args) {
    int[] nums = {0, 0, 1, 1, 1, 2, 2, 3, 3, 4};
    int[] result = Arrays.stream(nums).distinct().toArray();
    System.out.println("Input " + join(nums));
```

```
        System.out.println("Length " + result.length);
        System.out.println("Output " + join(result));
    }
}
```

Sample Output :

Input 0 0 1 1 1 2 2 3 3 4

Length 5

Output 0 1 2 3 4

Maximum Subarray

Aim :

To write a Java program to find the maximum sum of a contiguous subarray using reduce

Algorithm :

Step 1 Start the program

Step 2 Create an integer array nums

Step 3 Use reduction to maintain current sum and best sum

Step 4 Update current sum for every element

Step 5 Update best sum whenever current sum is greater

Step 6 Display the maximum subarray sum

Step 7 Stop the program

Program :

```
import java.util.Arrays;

public class Main {
   public static void main(String[] args) {
      int[] nums = {-2, 1, -3, 4, -1, 2, 1, -5, 4};
      int maxSum = Arrays.stream(nums).boxed()
            .reduce(new int[]{0, Integer.MIN_VALUE},
                  (acc, x) -> {
                     int current = Math.max(x, acc[0] + x);
                     int best = Math.max(acc[1], current);
                     return new int[]{current, best};
                  },
                  (a, b) -> a)[1];
```

```
        System.out.println("Maximum Subarray Sum " + maxSum);
    }
}
```

Sample Output :

Maximum Subarray Sum 6

Find the Highest Altitude

Aim :

To write a Java program to find the highest altitude reached from a gain array using reduce

Algorithm :

Step 1 Start the program

Step 2 Create an integer array gain

Step 3 Start altitude from zero

Step 4 Add each gain value to the current altitude

Step 5 Track the highest altitude

Step 6 Display the highest altitude

Step 7 Stop the program

Program :

```
import java.util.Arrays;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.stream.IntStream;

public class Main {
   public static void main(String[] args) {
      int[] gain = {-5, 1, 5, 0, -7};
      AtomicInteger altitude = new AtomicInteger(0);
      int highest = IntStream.concat(IntStream.of(0), Arrays.stream(gain).map(x -> altitude.addAndGet(x)))
            .max()
            .getAsInt();
      System.out.println("Highest Altitude " + highest);
```

```
    }
}
```

Sample Output :

Highest Altitude 1

Group Anagrams

Aim :

To write a Java program to group anagrams from a list of strings using stream pipeline processing

Algorithm :

Step 1 Start the program

Step 2 Create a string array words

Step 3 Sort the characters of each word to form a key

Step 4 Group words with the same key

Step 5 Convert grouped values into a list

Step 6 Display each group of anagrams

Step 7 Stop the program

Program :

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

public class Main {
   public static void main(String[] args) {
      String[] words = {"eat", "tea", "tan", "ate", "nat", "bat"};
      Map<String, List<String>> grouped = Arrays.stream(words)
            .collect(Collectors.groupingBy(word -> {
```

```
            char[] chars = word.toCharArray();
            Arrays.sort(chars);
            return new String(chars);
        }, LinkedHashMap::new, Collectors.toList()));
    List<List<String>> result = new ArrayList<>(grouped.values());
    result.forEach(group -> System.out.println(String.join(" ", group)));
  }
}
```

Sample Output :

eat tea ate

tan nat

bat

Top K Frequent Elements

Aim :

To write a Java program to find the top k most frequent elements from an array using stream pipeline processing

Algorithm :

Step 1 Start the program

Step 2 Create an integer array nums

Step 3 Store the value of k

Step 4 Count the frequency of every element

Step 5 Sort the elements in descending order of frequency

Step 6 Select the first k elements

Step 7 Display the result

Step 8 Stop the program

Program :

```
import java.util.Arrays;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

public class Main {
   public static void main(String[] args) {
      int[] nums = {1, 1, 1, 2, 2, 3};
      int k = 2;
      List<Integer> result = Arrays.stream(nums).boxed()
            .collect(Collectors.groupingBy(x -> x, Collectors.counting()))
```

```
            .entrySet()
            .stream()
            .sorted((a, b) -> Long.compare(b.getValue(), a.getValue()))
            .limit(k)
            .map(Map.Entry::getKey)
            .collect(Collectors.toList());
      System.out.println("Top Elements " +
result.stream().map(String::valueOf).collect(Collectors.joining(" ")));
   }
}
```

Sample Output :

Top Elements 1 2

Java Dequeue

Aim :

To write a Java program to find the maximum number of unique integers among all contiguous subarrays of size m using Deque and collection processing

Algorithm :

Step 1 Start the program

Step 2 Create an integer array nums

Step 3 Store the window size m

Step 4 Create a Deque to store the current window

Step 5 Create a HashMap to store element frequency

Step 6 Add each element to the deque and update frequency

Step 7 Remove the first element when the deque size becomes greater than m

Step 8 Update the maximum unique count

Step 9 Display the maximum number of unique integers

Step 10 Stop the program

Program :

```
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.HashMap;
import java.util.Map;

public class Main {
   public static void main(String[] args) {
      int[] nums = {5, 3, 5, 2, 3, 2};
      int m = 3;
```

```
        Deque<Integer> deque = new ArrayDeque<>();
        Map<Integer, Integer> frequency = new HashMap<>();
        int maxUnique = 0;
        for (int num : nums) {
            deque.addLast(num);
            frequency.put(num, frequency.getOrDefault(num, 0) + 1);
            if (deque.size() > m) {
                int removed = deque.removeFirst();
                frequency.put(removed, frequency.get(removed) - 1);
                if (frequency.get(removed) == 0) {
                    frequency.remove(removed);
                }
            }
            if (deque.size() == m) {
                maxUnique = Math.max(maxUnique, frequency.size());
            }
        }
        System.out.println("Maximum Unique Integers " + maxUnique);
    }
}
```

Sample Output :

Maximum Unique Integers 3

Java Hashset

Aim :

To write a Java program to count the number of unique name pairs after each insertion using HashSet

Algorithm :

Step 1 Start the program

Step 2 Create two string arrays to store first names and second names

Step 3 Create a HashSet to store unique pairs

Step 4 Combine each first name and second name as one pair

Step 5 Insert each pair into the set

Step 6 Display the size of the set after every insertion

Step 7 Stop the program

Program :

```
import java.util.HashSet;
import java.util.Set;

public class Main {
   public static void main(String[] args) {
      String[] firstNames = {"john", "john", "john", "mary", "mary"};
      String[] secondNames = {"tom", "mary", "tom", "anna", "anna"};
      Set<String> pairs = new HashSet<>();
      for (int i = 0; i < firstNames.length; i++) {
         pairs.add(firstNames[i] + " " + secondNames[i]);
         System.out.println(pairs.size());
      }
```

```
    }
}
```

Sample Output :

```
1
2
2
3
3
```